

CSE332

Computer Organization and Design

Chapter 6 & 7

Reference: Computer Systems Architecture

M. Morris Mano

**Slides are a reference of the book only. The course content will also extensively depend on the contents covered in the
*Class Lectures.***

Chapter 6

Assembly Language

MACHINE LANGUAGE

- **Program**

A list of instructions or statements for directing the computer to perform a required data processing task

- **Various types of programming languages**

- **Hierarchy of programming languages**

- **Machine-language**

- **Binary code**

- **Octal or hexadecimal code**

- **Assembly-language**

(Assembler)

- **Symbolic code**

- **High-level language**

(Compiler)

Comparison of Programming Language

BINARY PROGRAM TO ADD TWO NUMBERS:

Location	Instruction Code
0	0010 0000 0000 0100
1	0001 0000 0000 0101
10	0011 0000 0000 0110
11	0111 0000 0000 0001
100	0000 0000 0101 0011
101	1111 1111 1110 1001
110	0000 0000 0000 0000

HEXADECIMAL PROGRAM TO ADD TWO NUMBERS:

Location	Instruction
000	2004
001	1005
002	3006
003	7001
004	0053
005	FFE9
006	0000

PROGRAM WITH SYMBOLIC OPCODE:

Location	Instruction	Comments
000	LDA 004	Load 1st operand into
001	ADD 005	Add 2nd operand to AC
002	STA 006	Store sum in location 0
003	HLT	Halt computer
004	0053	1st operand
005	FFE9	2nd operand (negative)
006	0000	Store sum here

FORTRAN PROGRAM:

```
INTEGER A, B, C  
DATA A,83 / B,-23  
C = A + B  
END
```

ASSEMBLY LEVEL LANGUAGE

- The symbolic program (contains letters, numerals or special characters) is referred to as an assembly language program.
- The basic unit of an assembly language program is a line of code.
- Each line of assembly language program is arranged in three columns called fields:
 - The **label** field may be empty or it may contain a symbolic address of up to 3 characters.
 - The **instruction** field specifies a machine instruction or a pseudo instruction.
 - The **comment** field may be empty or it may include a comment.

INSTRUCTION FIELD

- Instruction field in an assembly level language may specify one of the following three items:
 - A memory-reference instruction(MRI) : MRI consist of two or three symbols separated by spaces.
 - ADD OPR (direct address MRI)
 - ADD PTR I (indirect address MRI)
 - A register-reference or input-output instruction(Non-MRI) : Non MRI instructions do not have an address part.
 - Pseudo instruction with or without operand : Symbolic address used in the instruction field must be defined somewhere as a label.

PSEUDO INSTRUCTION:

ASSEMBLER:

- Pseudo instruction is not a machine instruction but rather an instruction to the assembler giving information about some phase of translation.
- Four pseudo instructions that are recognized by an assembler are:
 - ORG N : Hexadecimal number N is the memory loc. for the instruction or operand listed in the **following** line.
 - END : Denotes the end of symbolic program.
 - DEC N : Signed decimal number N to be converted to binary.
 - HEX N : Hexadecimal number N to be converted to binary.

- An assembler is a program that accepts a symbolic language program and produces its binary machine language equivalent.
- The input symbolic program is called the source program and the output binary program is called the output program.
- The assembler is a program that operates on character strings and produces an equivalent binary interpretation.

Example: Assembly language program to subtract two numbers

	ORG 100	/ Origin of program is location 100
	LDA SUB	/ Load subtrahend to AC
	CMA	/ Complement AC
	INC	/ Increment AC
	ADD MIN	/ Add minuend to AC
	STA DIF	/ Store difference
	HLT	/ Halt computer
MIN,	DEC 83	/ Minuend
SUB,	DEC -23	/ Subtrahend
DIF,	HEX 0	/ Difference stored here
	END	/ End of symbolic program

TRANSLATION TO BINARY

<i>Hexadecimal Code</i>		<i>Symbolic Program</i>
<i>Location</i>	<i>Content</i>	
100	2107	ORG 100
101	7200	LDA SUB
102	7020	CMA
103	1106	INC
104	3108	ADD MIN
105	7001	STA DIF
106	0053	HLT
107	FFE9	MIN, SUB, DEC 83
108	0000	DIF, DEC -23
		HEX 0
		END

FIRST PASS OF AN ASSEMBLER:

Assembler

Source Program - Symbolic Assembly Language Program

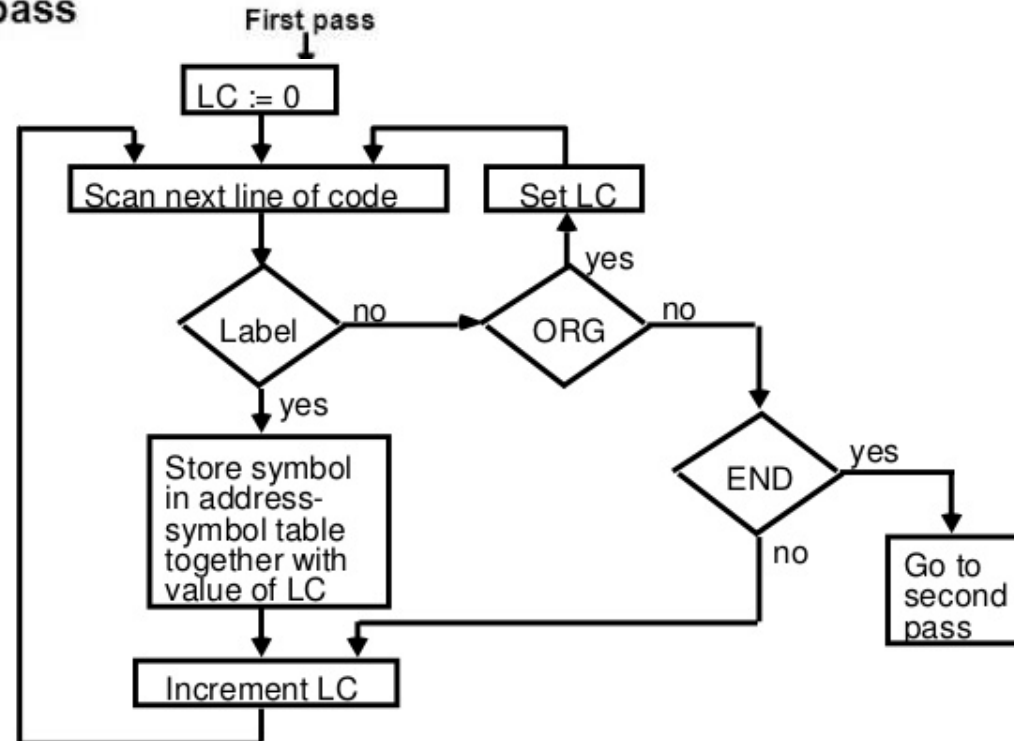
Object Program - Binary Machine Language Program

Two pass assembler

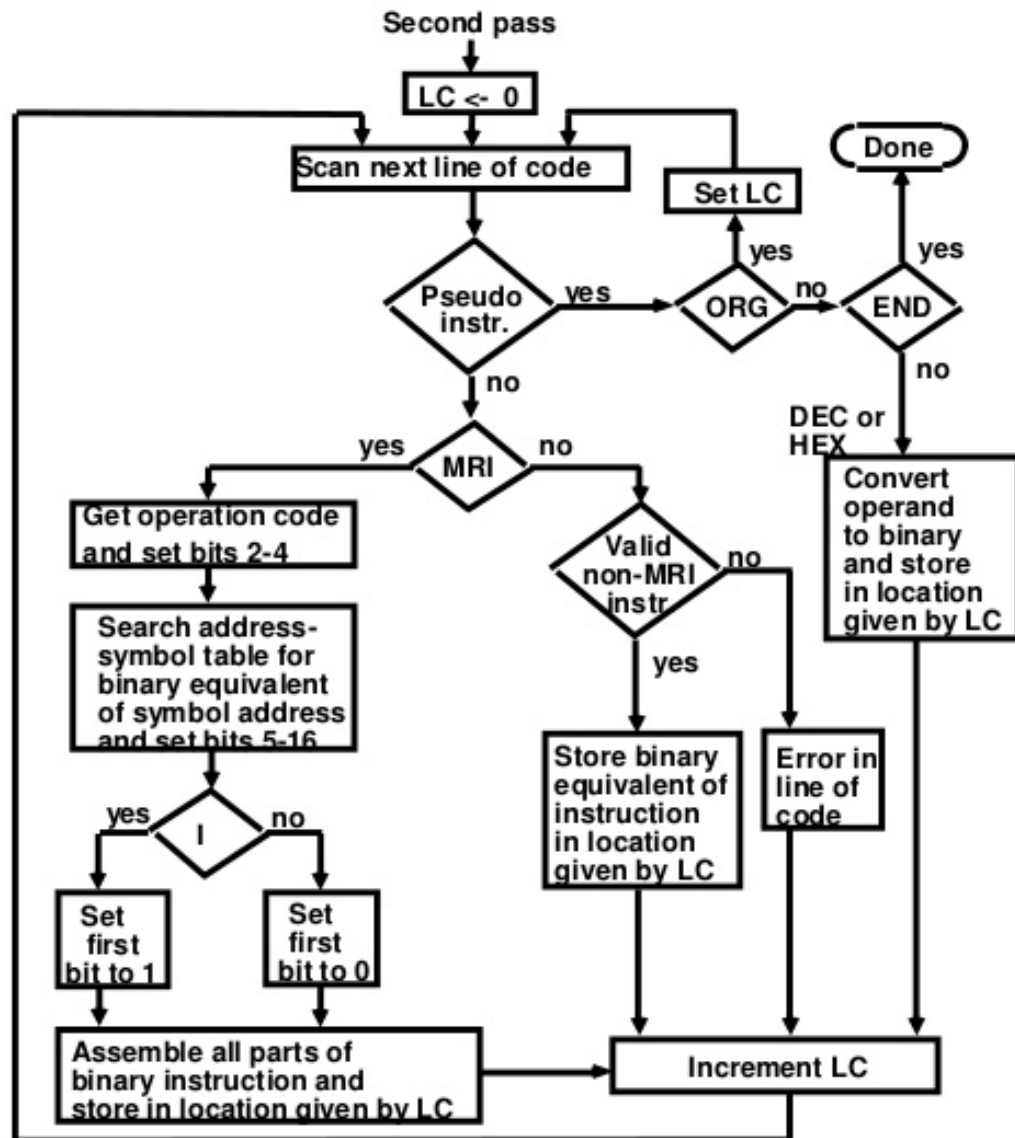
1st pass: generates a table that correlates all user defined (address) symbols with their binary equivalent value

2nd pass: binary translation

First pass



SECOND PASS



Second Pass

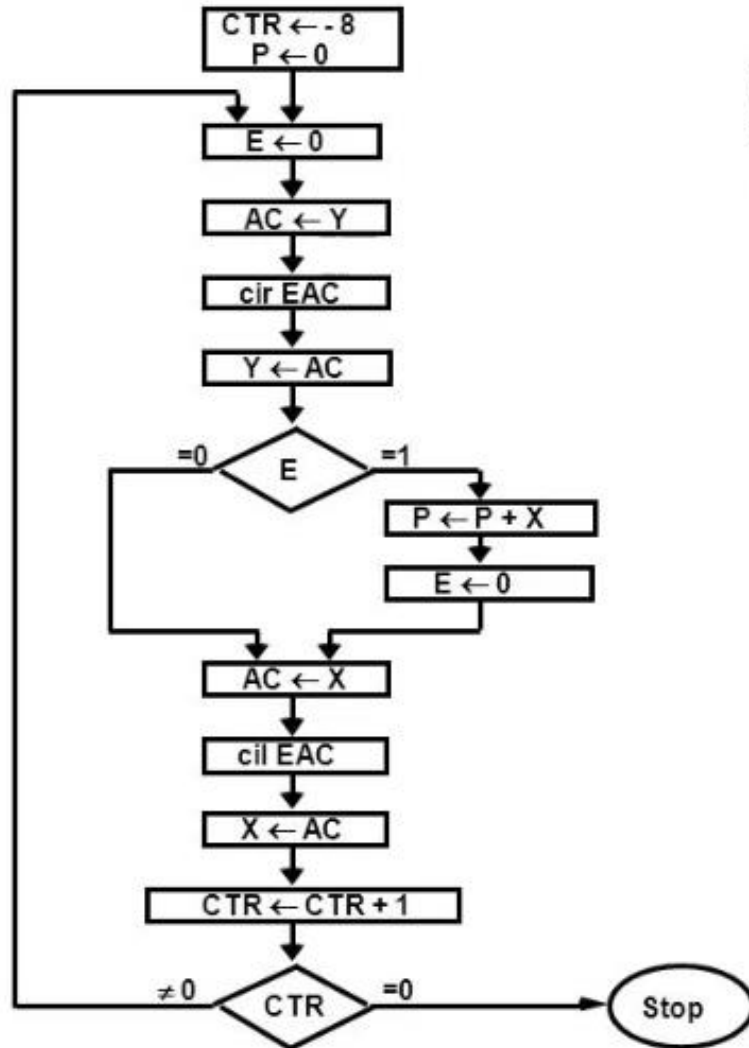
Machine instructions are translated by means of table-lookup procedures;
(1. Pseudo-Instruction Table, 2. MRI Table, 3. Non-MRI Table
4. Address Symbol Table)

PROGRAM LOOPS

Assembly-language program to add 100 numbers:

	ORG 100	/ Origin of program is HEX 100
	LDA ADS	/ Load first address of operand
	STA PTR	/ Store in pointer
	LDA NBR	/ Load -100
	STA CTR	/ Store in counter
	CLA	/ Clear AC
LOP,	ADD PTR I	/ Add an operand to AC
	ISZ PTR	/ Increment pointer
	ISZ CTR	/ Increment counter
	BUN LOP	/ Repeat loop again
	STA SUM	/ Store sum
	HLT	/ Halt
ADS,	HEX 150	/ First address of operands
PTR,	HEX 0	/ Reserved for a pointer
NBR,	DEC -100	/ Initial value for a counter
CTR,	HEX 0	/ Reserved for a counter
SUM,	HEX 0	/ Sum is stored here
	ORG 150	/ Origin of operands is HEX 150
	DEC 75	/ First operand
	:	
	:	
	DEC 23	/ Last operand
	END	/ End of symbolic program

Multiplication



X holds the multiplicand
Y holds the multiplier
P holds the product

Example with four significant digits

X =	0000 1111	P
Y =	0000 1011	0000 0000
	0000 1111	0000 1111
	0001 1110	0010 1101
	0000 0000	0010 1101
	0111 1000	1010 0101
	1010 0101	

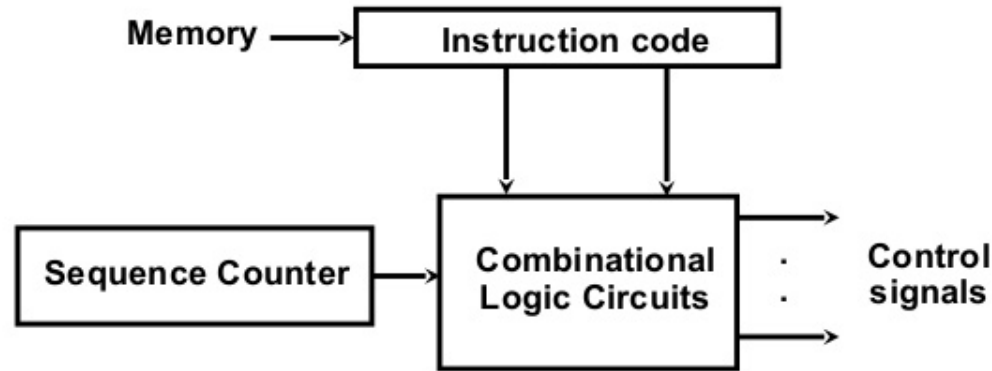
LOP,	ORG 100	
	CLE	/ Clear E
	LDA Y	/ Load multiplier
	CIR	/ Transfer multiplier bit to E
	STA Y	/ Store shifted multiplier
	SZE	/ Check if bit is zero
	BUN ONE	/ Bit is one; goto ONE
	BUN ZRO	/ Bit is zero; goto ZRO
ONE,	LDA X	/ Load multiplicand
	ADD P	/ Add to partial product
	STA P	/ Store partial product
	CLE	/ Clear E
ZRO,	LDA X	/ Load multiplicand
	CIL	/ Shift left
	STA X	/ Store shifted multiplicand
	ISZ CTR	/ Increment counter
	BUN LOP	/ Counter not zero; repeat loop
	HLT	/ Counter is zero; halt
CTR,	DEC -8	/ This location serves as a counter
X,	HEX 000F	/ Multiplicand stored here
Y,	HEX 000B	/ Multiplier stored here
P,	HEX 0	/ Product formed here
	END	

Chapter 7

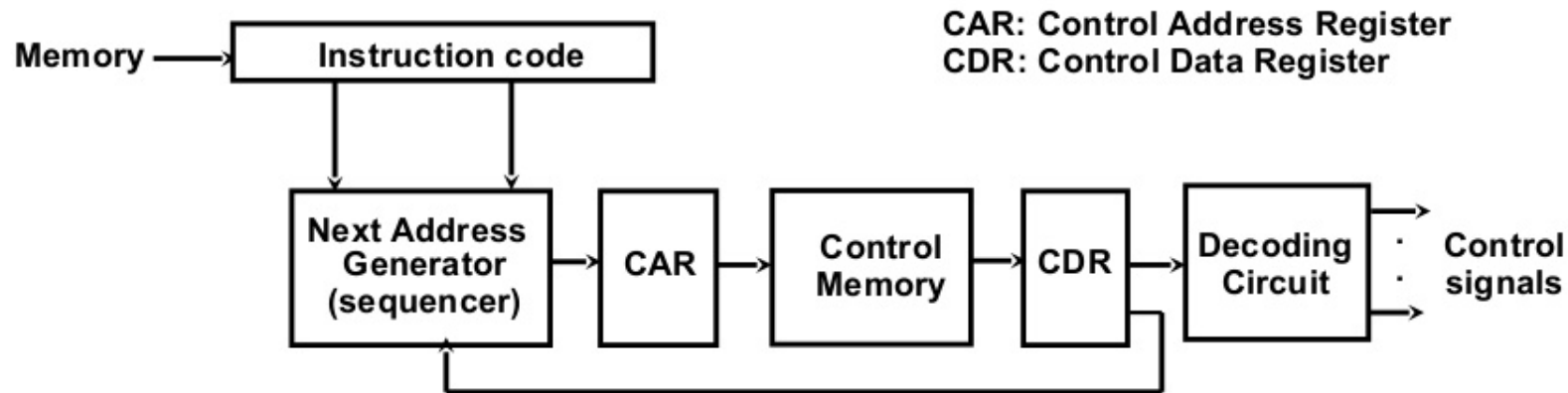
Microprogram Control

Control Unit Implementation

- Hardwired



- Microprogrammed

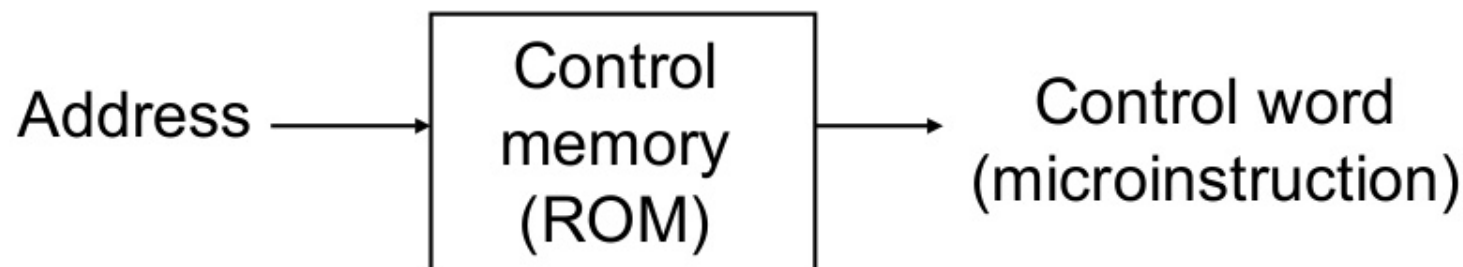


Microprogrammed Control Unit

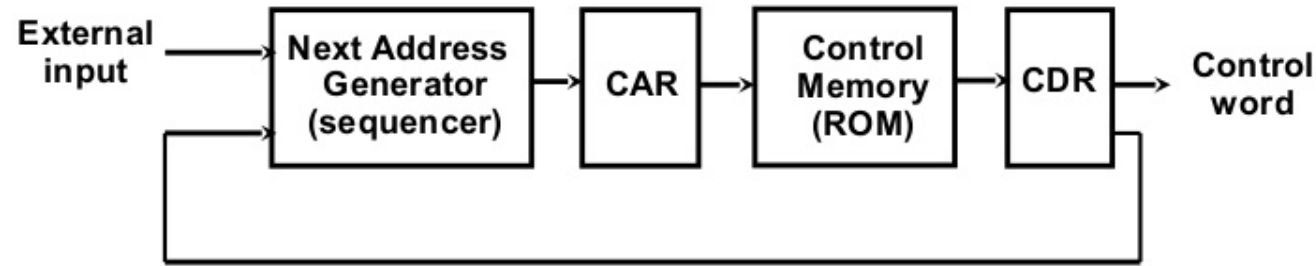
- Control signals
 - Group of bits used to select paths in multiplexers, decoders arithmetic logic units
- Control variables
 - Binary variables specify microoperations
 - Certain microoperations initiated while others idle
- Control word
 - String of 1's and 0's represent control variables
- Control memory
 - Memory contains control words
- Microinstructions
 - Control words stored in control memory
 - Specify control signals for execution of microoperations
- Microprogram
 - Sequence of microinstructions

Control Memory

- Read-only memory (ROM)
- Content of word in ROM at given address specifies microinstruction
- Each computer instruction initiates series of microinstructions (microprogram) in control memory
- These microinstructions generate microoperations to
 - Fetch instruction from main memory
 - Evaluate effective address
 - Execute operation specified by instruction
 - Return control to fetch phase for next instruction



Microprogrammed Control Organization

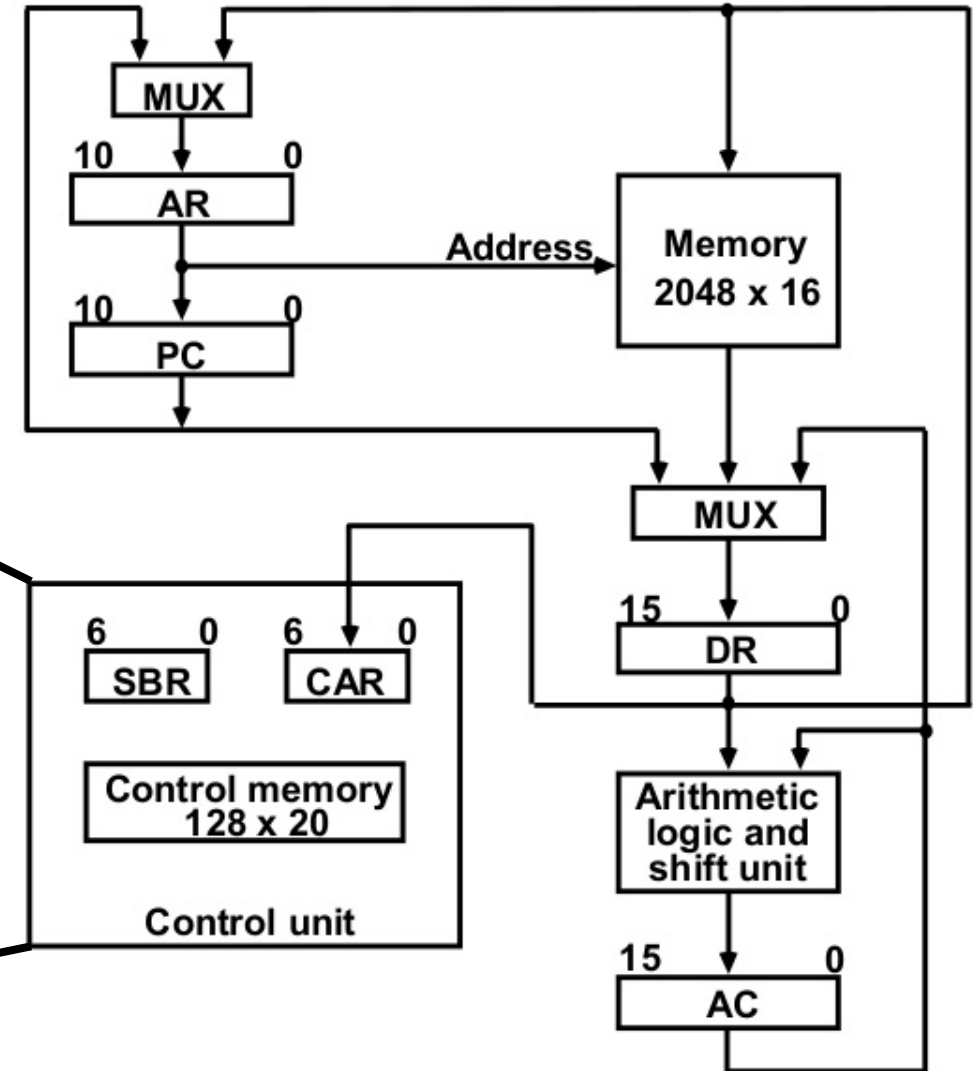
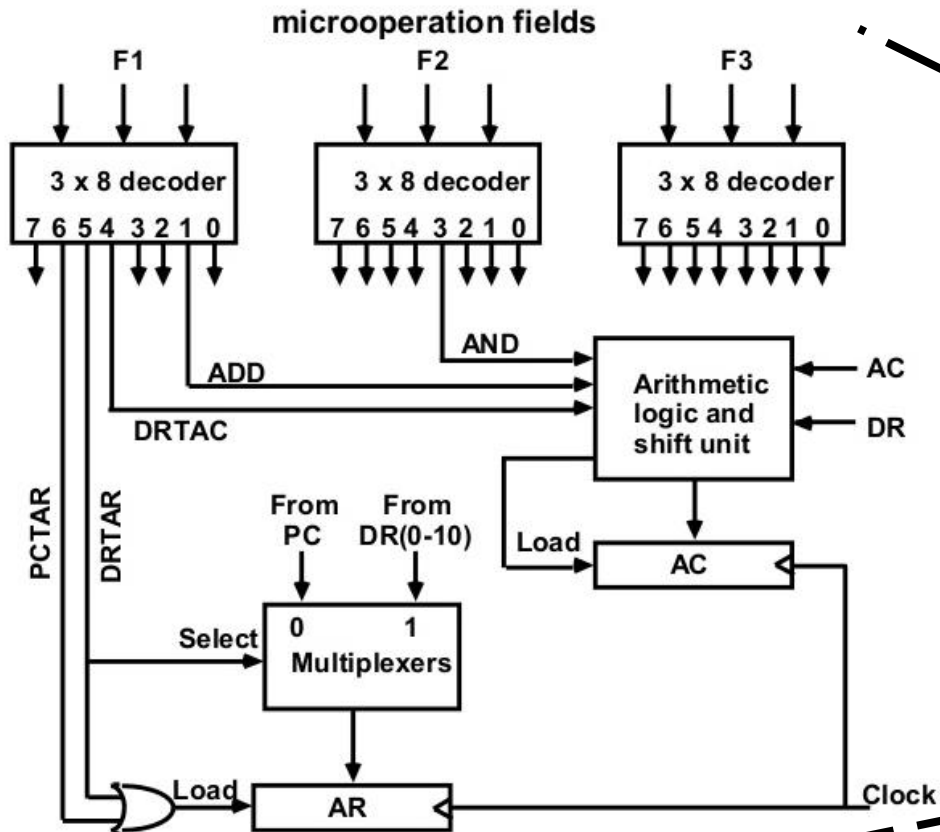


- Control memory
 - Contains microprograms (set of microinstructions)
 - Microinstruction contains
 - Bits initiate microoperations
 - Bits determine address of next microinstruction
- Control address register (CAR)
 - Specifies address of next microinstruction
- Next address generator (microprogram sequencer)
 - Determines address sequence for control memory
- Microprogram sequencer functions
 - Increment CAR by one
 - Transfer external address into CAR
 - Load initial address into CAR to start control operations
- Control data register (CDR)- or pipeline register
 - Holds microinstruction read from control memory
 - Allows execution of microoperations specified by control word simultaneously with generation of next microinstruction
- Subroutine register (SBR)
 - Stores return address during subroutine call

Microprogram Example

Design of Control Unit

Computer Configuration



Microinstruction Fields

Microinstruction Format

3	3	3	2	2	7
F1	F2	F3	CD	BR	AD

F1, F2, F3: Microoperation fields

CD: Condition for branching

BR: Branch field

AD: Address field

F1	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC + DR$	ADD
010	$AC \leftarrow 0$	CLRAC
011	$AC \leftarrow AC + 1$	INCAC
100	$AC \leftarrow DR$	DRTAC
101	$AR \leftarrow DR(0-10)$	DRTAR
110	$AR \leftarrow PC$	PCTAR
111	$M[AR] \leftarrow DR$	WRITE

F2	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC - DR$	SUB
010	$AC \leftarrow AC \vee DR$	OR
011	$AC \leftarrow AC \wedge DR$	AND
100	$DR \leftarrow M[AR]$	READ
101	$DR \leftarrow AC$	ACTDR
110	$DR \leftarrow DR + 1$	INCDR
111	$DR(0-10) \leftarrow PC$	PCTDR

CD	Condition	Symbol	Comments
00	Always = 1	U	Unconditional branch
01	DR(15)	I	Indirect address bit
10	AC(15)	S	Sign bit of AC
11	AC = 0	Z	Zero value in AC

F3	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC \oplus DR$	XOR
010	$AC \leftarrow AC'$	COM
011	$AC \leftarrow \text{shl } AC$	SHL
100	$AC \leftarrow \text{shr } AC$	SHR
101	$PC \leftarrow PC + 1$	INCP
110	$PC \leftarrow AR$	ARTPC
111	Reserved	

BR	Symbol	Function
00	JMP	$CAR \leftarrow AD$ if condition = 1 $CAR \leftarrow CAR + 1$ if condition = 0
01	CALL	$CAR \leftarrow AD, SBR \leftarrow CAR + 1$ if condition = 1 $CAR \leftarrow CAR + 1$ if condition = 0
10	RET	$CAR \leftarrow SBR$ (Return from subroutine)
11	MAP	$CAR(2-5) \leftarrow DR(11-14), CAR(0,1,6) \leftarrow 0$

Fetch Routine

- Fetch routine
 - Read instruction from memory
 - Decode instruction and update PC

Microinstructions for fetch routine:

$AR \leftarrow PC$
 $DR \leftarrow M[AR], PC \leftarrow PC + 1$
 $AR \leftarrow DR(0-10), CAR(2-5) \leftarrow DR(11-14), CAR(0,1,6) \leftarrow 0$

Symbolic microprogram for fetch routine:

FETCH: **ORG 64**
 PCTAR **U JMP NEXT**
 READ, INCPC **U JMP NEXT**
 DRTAR **U MAP**

Binary microporgram for fetch routine:

Binary address	F1	F2	F3	CD	BR	AD
1000000	110	000	000	00	00	1000001
1000001	000	100	101	00	00	1000010
1000010	101	000	000	00	11	0000000

Binary Microprogram

Micro Routine	Address		Binary Microinstruction					
	Decimal	Binary	F1	F2	F3	CD	BR	AD
ADD	0	0000000	000	000	000	01	01	1000011
	1	0000001	000	100	000	00	00	0000010
	2	0000010	001	000	000	00	00	1000000
	3	0000011	000	000	000	00	00	1000000
BRANCH	4	0000100	000	000	000	10	00	0000110
	5	0000101	000	000	000	00	00	1000000
	6	0000110	000	000	000	01	01	1000011
	7	0000111	000	000	110	00	00	1000000
STORE	8	0001000	000	000	000	01	01	1000011
	9	0001001	000	101	000	00	00	0001010
	10	0001010	111	000	000	00	00	1000000
	11	0001011	000	000	000	00	00	1000000
EXCHANGE	12	0001100	000	000	000	01	01	1000011
	13	0001101	001	000	000	00	00	0001110
	14	0001110	100	101	000	00	00	0001111
	15	0001111	111	000	000	00	00	1000000
FETCH	64	1000000	110	000	000	00	00	1000001
	65	1000001	000	100	101	00	00	1000010
	66	1000010	101	000	000	00	11	0000000
INDRCT	67	1000011	000	100	000	00	00	1000100
	68	1000100	101	000	000	00	10	0000000

Conditional Branching

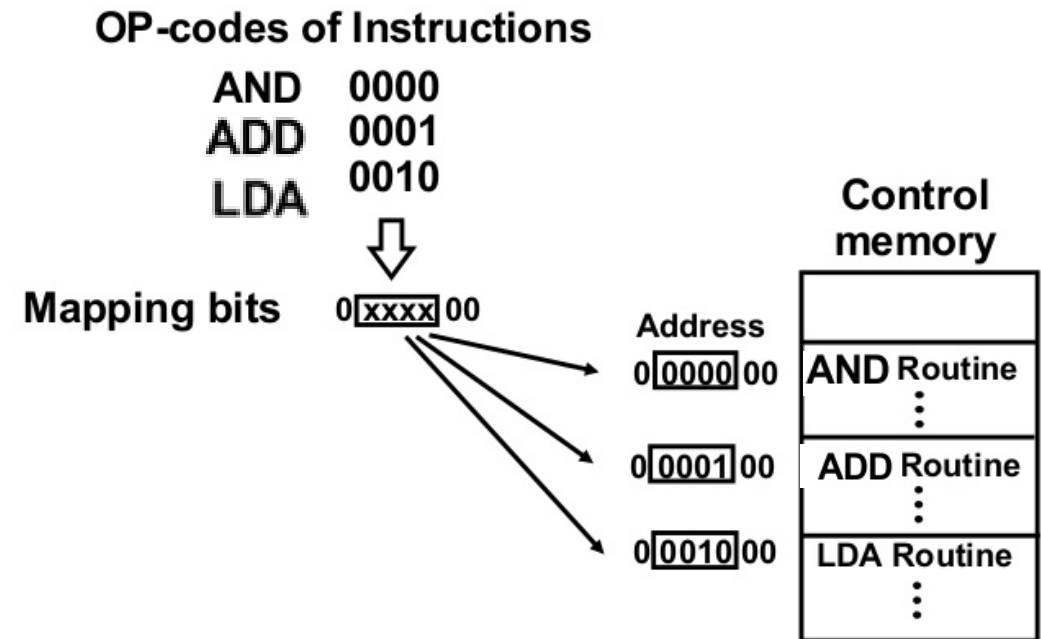
- Branching from one routine to another depends on status bit conditions
- Status bits provide parameter info such as
 - Carry-out of adder
 - Sign bit of number
 - Mode bits of instruction
- Info in status bits can be tested and actions initiated based on their conditions: 1 or 0
- Unconditional branch
 - Fix value of status bit to 1

Mapping of Instruction

- Each computer instruction has its own microprogram routine stored in a given location of the control memory
- Mapping
 - Transformation from instruction code bits to address in control memory where routine is located

Mapping of Instruction

- Example
 - Mapping 4-bit operation code to 7-bit address



Symbolic Microprogram

- Control memory: 128 20-bit words
- First 64 words: Routines for 16 machine instructions
- Last 64 words: Used for other purpose (e.g., fetch routine and other subroutines)
- Mapping: OP-code XXXX into 0XXXX00, first address for 16 routines are 0(0 0000 00), 4(0 0001 00), 8, 12, 16, 20, ..., 60

Partial Symbolic Microprogram

Label	Microops	CD	BR	AD
ADD:	ORG 0			
	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	ADD	U	JMP	FETCH
BRANCH:	ORG 4			
	NOP	S	JMP	OVER
	NOP	U	JMP	FETCH
	OVER:	I	CALL	INDRCT
STORE:	ARTPC	U	JMP	FETCH
	ORG 8			
	NOP	I	CALL	INDRCT
	ACTDR	U	JMP	NEXT
EXCHANGE:	WRITE	U	JMP	FETCH
	ORG 12			
	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
FETCH:	ACTDR, DRTAC	U	JMP	NEXT
	WRITE	U	JMP	FETCH
	ORG 64			
	PCTAR	U	JMP	NEXT
INDRCT:	READ, INCPC	U	JMP	NEXT
	DRTAR	U	MAP	
	READ	U	JMP	NEXT
	DRTAR	U	RET	